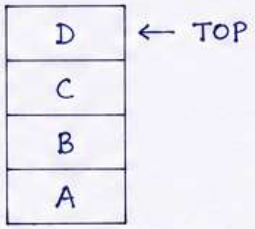
STACK AND ITS OPERATIONS

1) **STACK :** Stack is a linear data structure that follows LIFO (Last In First Out) order. The element which is inserted last is deleted first. Insertion and Deletion are allowed at one end only which is called TOP.

Example :

Suppose we insert A, B, C, D in stack. Then D is at top. It will be deleted first.



2) **OPERATIONS ON STACK :**

The following are the basic operations.

- PUSH
- POP
- PEEK (or TOP)
- ISEMPTY
- ISFULL

TERMS :

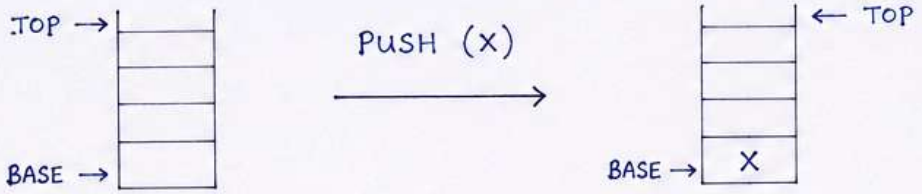
TOP → Points to top element of stack.

BASE → Bottom of stack.

SIZE → Maximum size of stack.

3) **PUSH OPERATION :**

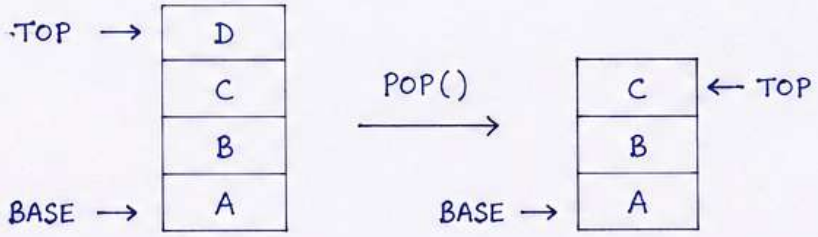
Push operation is used to insert an element in stack. New element is inserted at TOP. After insertion, TOP is updated.



(Before Push) (After Push)

4) **POP OPERATION :**

Pop operation is used to delete an element from stack. Element at TOP is deleted and TOP is updated.



(Before Pop) (After Pop)

5) **PEEK (or TOP) OPERATION :**

Peek operation is used to return the element at TOP without deleting it.
i.e. It only views the top element.

Example : If stack is [A, B, C, D] then PEEK() will return D.

6) **ISEMPTY OPERATION :**

This operation checks whether stack is empty or not.
If TOP = -1 (or stack has no element)
then stack is empty, otherwise not empty.

7) **ISFULL OPERATION :**

This operation checks whether stack is full or not.
If TOP = SIZE - 1
then stack is full, otherwise not full.

NOTE : All operations (PUSH, POP, PEEK, ISEMPTY, ISFULL) take O(1) time.

MULTIPLE STACKS

1) INTRODUCTION :

In many applications, we need more than one stack.
Multiple stacks mean maintaining more than one stack in a single array.
This helps in efficient memory utilization.

2) TYPES OF MULTIPLE STACKS :

There are mainly two types :

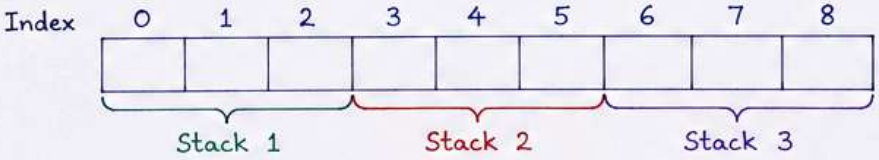
a) Fixed Size Multiple Stacks

b) Variable Size Multiple Stacks

3) FIXED SIZE MULTIPLE STACKS :

In this approach, the array is divided into equal parts and each part is used as a separate stack.

Example : Suppose we want to implement 3 stacks in an array of size 9.



Each stack can store maximum 3 elements.
If any stack exceeds its limit, it results in OVERFLOW even if other stacks have free space.

4) VARIABLE SIZE MULTIPLE STACKS :

In this approach, all stacks share the array space.
The stacks can grow and shrink dynamically.
If one stack grows, it uses free space from other stacks.

Two methods to implement :

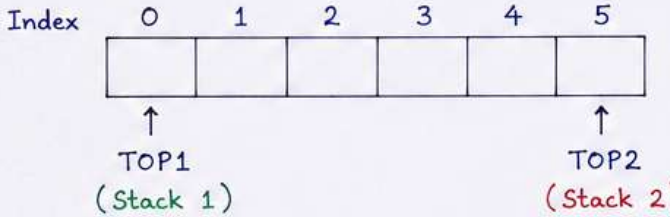
a) Two in One Approach (for 2 stacks)

b) K in N Approach (for K stacks in n size array)

a) TWO IN ONE APPROACH (Two Stacks in One Array) :

One stack grows from left to right and the other grows from right to left.
Initially, TOP1 = -1 and TOP2 = N.
If TOP1 < TOP2 - 1, there is space for insertion.

Example : Array size = 6



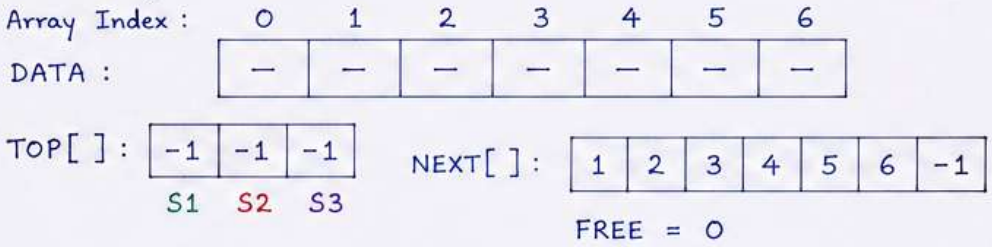
Condition for Overflow :
TOP1 >= TOP2 - 1

b) K IN N APPROACH (k Stacks in an Array of Size n) :

We maintain two extra arrays.

- TOP[k] : stores the top index of each stack.
- NEXT[n] : stores the next index for each element.
- FREE : stores the index of next free location.

Example : Implementing 3 stacks in an array of size 7.
Initially :



5) ADVANTAGES :

- Efficient memory utilization.
- Useful in applications where number of stacks is fixed.
- Avoids wastage of memory.

NOTE : Multiple stacks save space and improve efficiency when compared to using separate arrays for each stack.

APPLICATIONS OF STACK FOR EXPRESSION CONVERSION

1) EXPRESSION CONVERSION :

Stack is widely used to convert expressions from one notation to another.

The three common notations are :

Infix	:	A + B
Postfix	:	A B +
Prefix	:	+ A B

2) INFIX TO POSTFIX CONVERSION :

A stack is used to temporarily store operators and parentheses.

Example : A + B * C

Postfix : A B C * +

Steps :

1. Scan the expression from left to right.
2. Operands are directly added to the output.
3. Operators are pushed into the stack according to precedence.
4. Parentheses are handled using the stack.
5. At the end, remaining operators are popped from the stack and added to output.

3) INFIX TO PREFIX CONVERSION :

Stack is also used to convert an infix expression into prefix notation.

Example : Infix : A + B * C

Prefix : + A * B C

SUMMARY : Stack → Expression Conversion → Evaluation

Infix → Postfix / Prefix → Evaluation

4) POSTFIX EXPRESSION EVALUATION :

A stack is used to evaluate postfix expressions.

Example : 2 3 + 4 *

Token	Operation	Stack (bottom → top)
2	Push 2	2
3	Push 3	2, 3
+	2 + 3 = 5, Push 5	5
4	Push 4	5, 4
*	5 * 4 = 20, Push 20	20

Result = 20

5) PREFIX EXPRESSION EVALUATION :

Stack is used to evaluate prefix expressions by scanning the expression from right to left.

Example : * + 2 3 4

Result : (2 + 3) x 4 = 20

6) PARENTHESIS MATCHING :

Stack helps in checking whether parentheses are balanced and correctly arranged.

Example : [A + (B - C)]

Opening brackets are pushed into stack and removed when their corresponding closing brackets are encountered.

7) OTHER APPLICATIONS :

- Compiler expression parsing
- Arithmetic expression evaluation
- Scientific calculators
- Programming language interpreters
- Syntax checking
- Expression parsing in compilers

MAIN REASON :

Stack follows LIFO (Last In First Out) order, which is suitable for handling operators, precedence, parentheses and intermediate results.

QUEUES : QUEUE OPERATIONS

1) QUEUE :

A queue is a linear data structure that follows FIFO (First In First Out) order.
Insertion is done at REAR end and Deletion is done from FRONT end.

2) BASIC OPERATIONS ON QUEUE :

The following are the basic operations.

- ENQUEUE (Insert)
- DEQUEUE (Delete)
- PEEK (or FRONT)
- ISEMPTY
- ISFULL

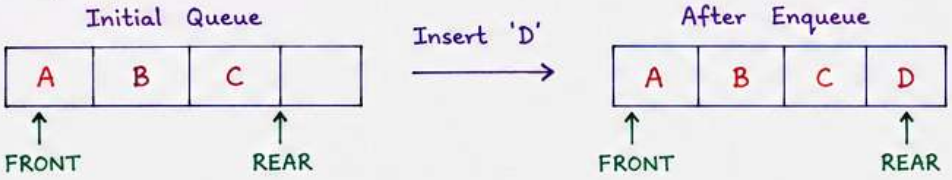
3) ENQUEUE (INSERT) :

Enqueue operation is used to insert an element at REAR end of the queue.

Steps :

① Check if queue is full. If full, report OVERFLOW.
② Otherwise, increment REAR.
③ Insert the new element at REAR.

Example :



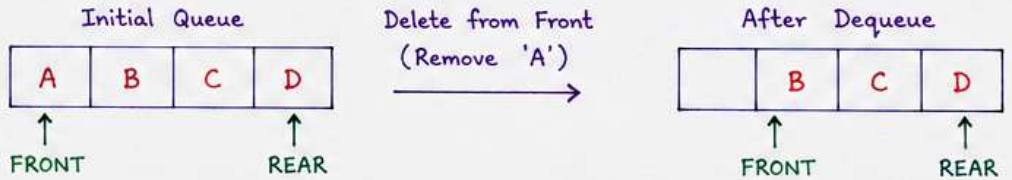
4) DEQUEUE (DELETE) :

Dequeue operation is used to delete an element from FRONT end of the queue.

Steps :

① Check if queue is empty. If empty, report UNDERFLOW.
② Otherwise, delete the element at FRONT.
③ Increment FRONT.

Example :



5) PEEK (or FRONT) :

Peek operation is used to get the element at FRONT without deleting it.

Step : If queue is empty, report UNDERFLOW, otherwise return element at FRONT.

6) ISEMPTY :

This operation checks whether the queue is empty or not.
If queue is empty, return TRUE otherwise return FALSE.

7) ISFULL :

This operation checks whether the queue is full or not.
If queue is full, return TRUE otherwise return FALSE.

SUMMARY :

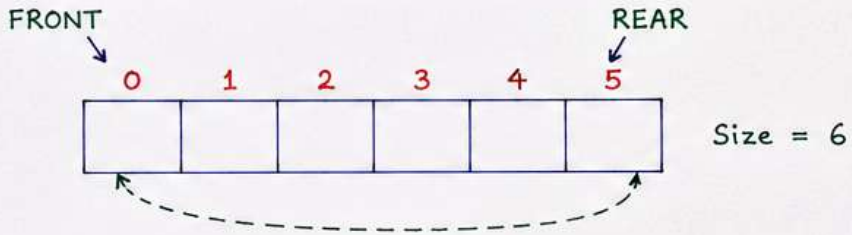
- Enqueue → Insert at REAR
- Dequeue → Delete from FRONT
- Peek → View FRONT element
- isEmpty → Check if empty
- isFull → Check if full

NOTE :
All operations take O(1) time.

Circular Queue Its Advantages and Applications

1) Circular Queue :

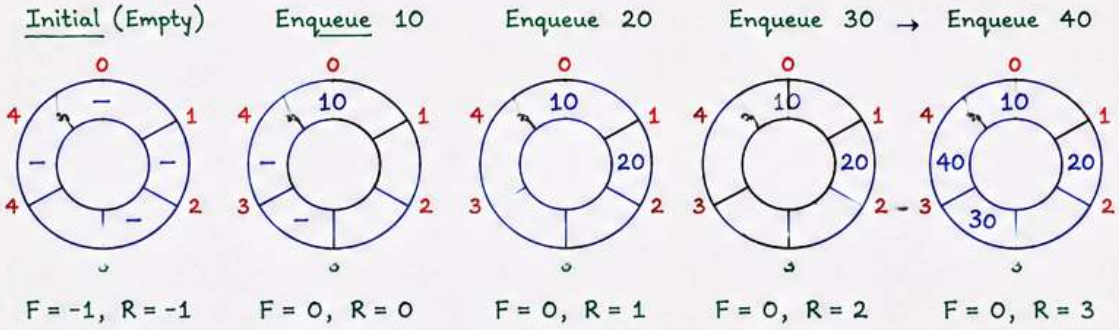
Circular Queue is a linear data structure in which the last position is connected back to the first position to form a circle.
It follows FIFO (First In First Out) order.



2) Working :

When REAR reaches the last index (n-1), the next position will be 0.
Similarly, FRONT also moves circularly.

Example (Size = 5)



3) Queue Full Condition :

Queue is full when

$$(REAR + 1) \% N = FRONT$$

where N is the size of queue.

4) Queue Empty Condition :

Queue is empty when

$$FRONT = -1$$

(or) FRONT > REAR (in some cases)

5) Advantages :

1. Efficient memory utilization.
   In normal queue, once REAR reaches end, no more insertions are possible even if there is space at beginning. Circular queue solves this problem.
2. No wastage of space.
3. Better performance.
4. Useful for fixed size data structures.
5. Easy to implement using array.

6) Applications :

1. CPU Scheduling (Round Robin Scheduling)
2. Handling of I/O buffers in operating systems.
3. Printer Spooling.
4. Simulation of real life queues (ticket booking, waiting lines, etc.)
5. Data communication (circular buffers).
6. Used in network routers and router queues.
7. Memory management.
8. Used in games and multimedia applications.
9. Any situation where continuous queue operations are required with fixed size.

NOTE : Circular Queue is more efficient than linear queue because it overcomes the problem of false overflow.

Priority Queue and Its Advantages and Applications

1) Priority Queue :

A Priority Queue is an abstract data type in which each element has a priority.
Elements are removed based on their priority.
The element with highest priority is deleted first (in max-priority queue).
If two elements have same priority, then they are removed in FIFO order.

> It is a combination of Queue and Priority.

2) Types :

- Max-Priority Queue → Element with highest priority is served first.
- Min-Priority Queue → Element with lowest priority is served first.

3) Operations :

Operation	Description
insert(x, p)	Inserts element x with priority p.
delete()	Deletes and returns element with highest (or lowest) priority.
peek()	Returns element with highest (or lowest) priority without deleting it.
isEmpty()	Checks whether the priority queue is empty.
isFull()	Checks whether the priority queue is full.

4) Example (Max-Priority Queue) :

Insert (element, priority)	Priority Queue (Front)	delete() operation
insert (A, 2) →	B(5)	delete() → B(5)
insert (B, 5) →	E(4)	delete() → E(4)
insert (C, 3) →	A(2)	delete() → C(3)
insert (D, 1) →	D(1)	delete() → A(2)
insert (E, 4) →	(Rear)	delete() → D(1)

Note : Elements are served in decreasing order of priority (highest priority first).

5) Advantages :

1. Important elements are processed first.
2. Efficient handling of urgent tasks.
3. Helps in optimizing CPU scheduling and resource allocation.
4. Can be implemented using various data structures like Heap for better performance.
5. Useful in situations where priority is more important than arrival time.

6) Applications :

1. CPU Scheduling in Operating Systems.
2. Dijkstra's shortest path algorithm in Graph.
3. Job scheduling in real-time systems.
4. Event-driven simulation systems.
5. Managing interrupts in embedded systems.
6. Bandwidth management in networks.
7. Data compression algorithms (Huffman Coding).

Implementation :
Priority Queue can be implemented using :

- Arrays (simple implementation)
- Linked Lists (sorted by priority)
- Heaps (Binary Heap / Binary Min-Heap / Max-Heap)
  - Most efficient implementation.

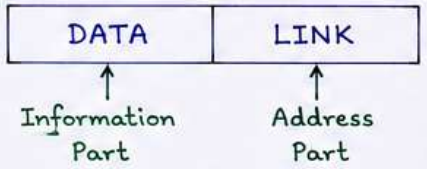
Note : Priority Queue is very useful when we need to process elements based on priority and not in the order they arrive.

Linked List : Introduction of Linked Lists

1) Introduction :

- A Linked List is a linear data structure.
- It consists of a sequence of nodes.
- Each node contains two parts :
  - → Data (Information part)
  - → Link (Address part)

Node Structure :

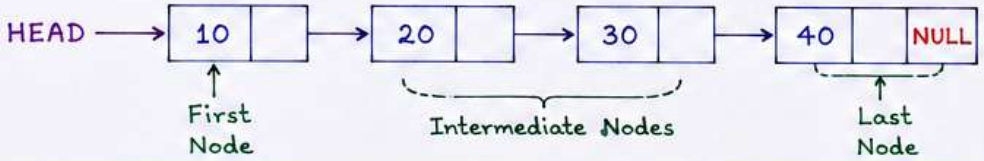


LINK field stores the address of the next node in the list.

2) Why Linked List ?

- In arrays, size is fixed and cannot be changed.
- In Linked List, size is dynamic (can grow or shrink at run time).
- Efficient insertion and deletion of nodes.
- Does not require contiguous memory locations.

3) Representation :



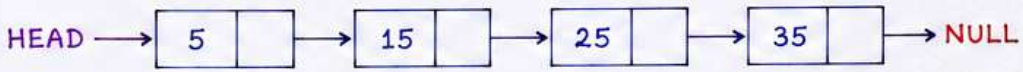
4) Types of Linked Lists :

1. Singly Linked List
2. Doubly Linked List
3. Circular Singly Linked List
4. Circular Doubly Linked List

5) Singly Linked List (Basic Form) :

- Each node points to the next node.
- Last node's link part contains NULL.
- Traversal is possible in one direction only (forward).

Example :



6) Terms :

- Head : Pointer to the first node of the list.
- Node : Basic element of linked list.
- Link : Address of the next node.
- NULL : Special value indicating the end of the list.
- Traversal : Process of visiting each node in the list.

Advantages :

- → Dynamic size.
- → Efficient insertion and deletion.
- → Easy to implement various data structures like stacks, queues, graphs, etc.
- → Better memory utilization.

Disadvantages :

- → Extra memory required for link part.
- → Random access is not allowed (no indexing).
- → Traversal is slower than arrays.

Applications :

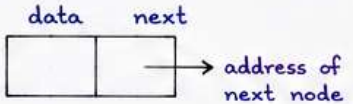
- → Implementation of stacks, queues.
- → Polynomial representation.
- → Sparse matrix representation.
- → Graph representation (Adjacency List).
- → Memory management.

Note : Linked List is a dynamic data structure where elements (nodes) are stored in non-contiguous memory locations and linked together using pointers.

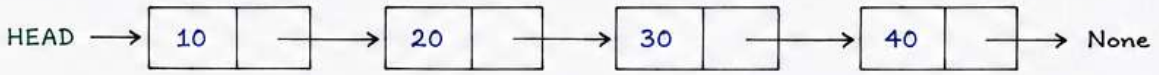
PRIMITIVE OPERATIONS ON LINKED LIST – Create, Traverse, Search, Insert, Delete, Sort, and Concatenate (USING PYTHON)

Node Structure

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```



Example Linked List : 10 → 20 → 30 → 40 → None



1) CREATE (At End)

```
def create(head, data):
    new_node = Node(data)          # Create a new node
    if head is None:               # If list is empty,
        return new_node            #   new node becomes head
    temp = head                    # Move to last node
    while temp.next:
        temp = temp.next
    temp.next = new_node           # Link new node at end
    return head                    # Return head
```

2) TRAVERSE (Display)

```
def traverse(head):
    temp = head                    # Start from head
    if head is None:
        print("List is empty")     # Check empty list
        return
    while temp:
        print(temp.data, end=' → ') # Visit each node
        temp = temp.next           #   and move to next
    print("None")                  # End of list
```

3) SEARCH

```
def search(head, key):
    temp = head                    # Start from head
    pos = 1
    while temp:                    # Compare key
        if temp.data == key:       # If found, return
            return pos             #   position
        temp = temp.next           # Move to next node
        pos += 1
    return -1  # Not found         # Not found
```

4) INSERT (At Position)

```
def insert(head, data, pos):
    new_node = Node(data)          # New node
    if pos == 1:                   # Insert at beginning
        new_node.next = head
        return new_node
    temp = head
    for i in range(1, pos-1):      # Move to (pos-1)th
        if temp is None:           #   node
            return head  #Invalid position
        temp = temp.next
    new_node.next = temp.next      # Link new node
    temp.next = new_node           # Return head
    return head
```

Example : Insert 25 at position 3 in 10→20→30→40

Result : 10→20→25→30→40

5) DELETE (At Position)

```
def delete(head, pos):
    if head is None:
        return None
    if pos == 1:
        return head.next    # Delete first node
    temp = head
    for i in range(1, pos-1):
        if temp.next is None:
            return head     # Invalid position
        temp = temp.next
    if temp.next is None:
        return head     # Position > length
    temp.next = temp.next.next  # Bypass node
    return head
```

Example : Delete node at position 3 from 10→20→30→40

Result : 10→20→40

6) SORT (Ascending Order)

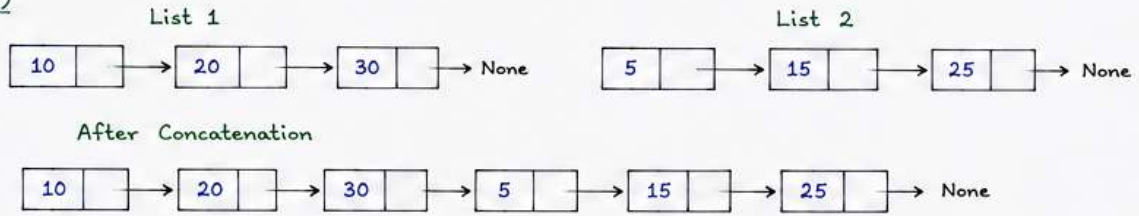
```
def sort(head):
    if head is None or head.next is None:
        return head
    for i in range(head):
        temp = head                # Bubble sort
        while temp.next:           #   by swapping
            if temp.data > temp.next.data:  #   data
                temp.data, temp.next.data = \
                    temp.next.data, temp.data
            temp = temp.next
    return head
```

Example : 30→10→40→20

Sorted : 10→20→30→40

7) CONCATENATE (Join Two Linked Lists)

```
def concatenate(head1, head2):
    if head1 is None:              # If list1 empty,
        return head2               #   return list2
    temp = head1                   # Move to last node of list1
    while temp.next:
        temp = temp.next
    temp.next = head2              # Link last node of list1
    return head1                   #   to head of list2
                                   # Return head of combined list
```



★ These are basic operations performed on Singly Linked List.

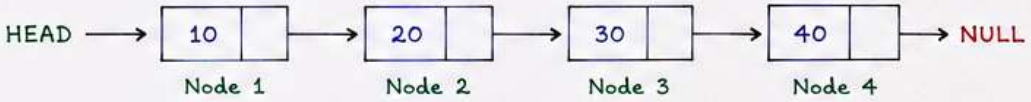
NOTE : All operations (except traverse) return the updated head of the list.

TYPES OF LINKED LIST

Linked List is a linear data structure in which elements (nodes) are not stored in contiguous memory locations. Each node contains data and link (address) to the next node.

1) SINGLY LINKED LIST (Linear)

Each node contains two parts :
- data : stores information
- next : stores address of next node



Python Node Class :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

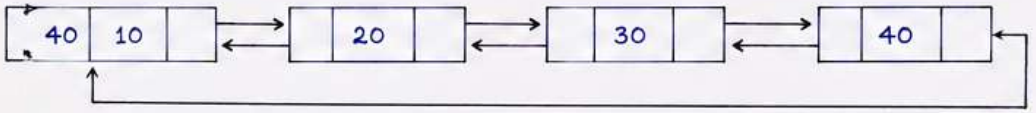
Features :
- Traversal possible in one direction only.
- Last node's next contains NULL.
- Memory efficient.

2) CIRCULAR LINKED LIST

In circular linked list, the last node's next field points back to the first node (head).

a) Circular Singly Linked List

b) Circular Doubly Linked List

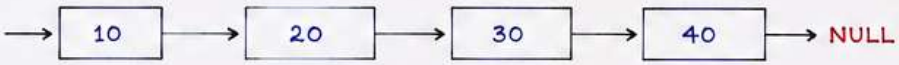


Features :
- Last node is connected back to the first node.
- No NULL at the end.
- Useful for circular data processing.

3) LINEAR LINKED LIST

Linear linked list is also called as singly linked list (as above).
Nodes are arranged in a sequence and last node points to NULL.
(Basically same as Singly Linked List)

Diagram :



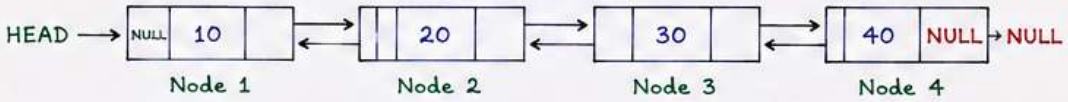
Features :
- Linear arrangement of nodes.
- Last node's next contains NULL.
- Easy implementation.
- Used widely in stacks, queues (using linked list), etc.

Python Node Class :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

4) DOUBLY LINKED LIST

Each node contains three parts :
- prev : stores address of previous node
- data : stores information
- next : stores address of next node



Python Node Class :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Features :
- Traversal possible in both directions.
- Extra memory for prev pointer.
- Useful for operations requiring backward traversal.